

**FACULTY OF APPLIED SCIENCE
DEPARTMENT OF COMPUTER SCIENCE**



NAME : JOSHUA NYONI

STUDENT NO : N02019084G

DEGREE PROGRAMME : BACHELOR DEGREE IN COMPUTER SCIENCE

MODULE : PARALLEL AND DISTRIBUTED COMPUTING

COURSE CODE : SCS4212

LECTURER : S NDLOVU

QUESTION 1: AS WITH ANY SOFTWARE DESIGN, PERFORMANCE IS AN ISSUE IN THE DESIGN OF A DISTRIBUTED APPLICATION. THE DESCRIPTION OF THE LIGHT WEIGHT RPC SHOWS SOME OPTIMIZATION THAT MAY BE ADOPTED FOR BETTER PERFORMANCE OF DISTRIBUTED APPLICATION USING RPC. OUTLINE OTHER OPTIMIZATION THAT MAY ALSO HAVE A SIGNIFICANT PAYOFF WHEN ADOPTED FOR DESIGNING RPC-BASED DISTRIBUTED APPLICATIONS.

SOLUTION

Several strategies can be adopted to optimize the performance of RPC-based distributed applications beyond the optimizations provided by Lightweight RPC (LRPC), these include:

- **Concurrent Access to Multiple Servers:** Utilize threads in the client process to independently make RPC calls to different servers. This requires a protocol that supports rich addressing for correct routing.
- **Early Reply Method:** Implement a method where an RPC call is split into two parts: one for passing parameters to the server and another for requesting the result. The server returns a tag with the first call, which is sent back with the second call to match the result. This allows the client to decide the time delay between calls and perform other activities during this period, potentially making several other RPC calls. However, this method can lead to congestion or unnecessary overhead if the request for results is delayed
- **Serving Multiple Requests Simultaneously:** Employ multithreading to process other requests while a server waits for a resource that is momentarily unavailable. This can help avoid delays caused by a server invoking a remote function that requires significant computation or has a high transmission latency
- **Reducing Per-Call Workload of Servers:** Keep requests brief and minimize the work a server has to do for each request. Use stateless servers to avoid being tied to any one state, which can improve performance when handling a large number of client requests
- **Reply Caching of Idempotent Remote Procedures:** Utilize reply caching for sending responses when the server cannot handle client requests due to the difference in pace. This helps manage backlogs caused by unhandled client requests at the same pace
- **Careful Choice of Timeout Values:** Select appropriate timeout values to avoid unnecessary retransmissions due to timeouts that are too small or too large, which can lead to delays or excessive network traffic
- **Appropriate Design of RPC Protocol Specifications:** Design the protocol specifications to reduce the amount of data transferred over the network and the frequency with which it is sent. This can include optimizing data structures and algorithms used in the protocol
- **Design for Concurrency:** Use multiple processors with shared memory to achieve high performance in terms of high call throughput and low call latency. Reduce lock contention and the utilization of shared-data structures to increase throughput, and decrease the overhead of context switching to lower latency

By adopting these optimizations, RPC-based distributed applications can achieve better performance, ensuring efficient and effective communication between distributed components

References

Waldo J, Wyant G, Wollrath A and Kendall S.(1994). A note on distributed computing. Sun Microsystems laboratories technical report series. M/S 29-01